

Module-4 **Logic and Reasoning**

Introduction to Logic and Reasoning-Propositional Logic-First Order Logic-Inference in First Order Logic-Unification, Forward Chaining, Backward Chaining, Resolution.

First-Order Logic:

- Atomic and Complex Sentences
- Quantifiers
- Converting sentences into logical form
- Inference
- Unification
- Resolution
- Modus Ponens
- Horn Clause and Definite clause
- Forward Chaining
- Backward Chaining.

First-order logic

- First-order logic (FOL) models the world in terms of
 - **Objects**, which are things with individual identities
 - **Properties** of objects that distinguish them from others
 - **Relations** that hold among sets of objects
 - **Functions**, which are a subset of relations where there is only one “value” for any given “input”
- Examples:
 - Objects: Students, lectures, companies, cars ...
 - Relations: Brother-of, bigger-than, outside, part-of, has-color, occurs-after, owns, visits, precedes, ...
 - Properties: blue, oval, even, large, ...
 - Functions: father-of, best-friend, second-half, more-than ...

User provides

- **Constant symbols** representing individuals in the world
 - Mary, 3, green
- **Function symbols**, map individuals to individuals
 - father_of(Mary) = John
 - color_of(Sky) = Blue
- **Predicate symbols**, map individuals to truth values
 - greater(5,3)
 - green(Grass)
 - color(Grass, Green)

FOL Provides

- **Variable symbols**

- E.g., x , y , foo

- **Connectives**

- Same as in propositional logic: not ($\neg$), and ($\wedge$), or ($\vee$), implies ($\rightarrow$), iff ($\leftrightarrow$)

- **Quantifiers**

- Universal $\forall \mathbf{x}$ or $(\mathbf{Ax})$

- Existential $\exists \mathbf{x}$ or $(\mathbf{Ex})$

Sentences: built from terms and atoms

- A **term** (denoting a real-world individual) is a constant symbol, variable symbol, or n-place function of n terms.
 - x and $f(x_1, \dots, x_n)$ are terms, where each x_i is a term
 - A term with no variables is a **ground term** (i.e., `john`, `father_of(father_of(john))`)
- An **atomic sentence** (which has value true or false) is an n-place predicate of n terms (e.g., `green(Kermit)`)
- A **complex sentence** is formed from atomic sentences connected by the logical connectives:
 - $\neg P$, $P \vee Q$, $P \wedge Q$, $P \rightarrow Q$, $P \leftrightarrow Q$ where P and Q are sentences

Sentences: built from terms and atoms

- A **quantified sentence** adds quantifiers $\forall$ and $\exists$
- A **well-formed formula (wff)** is a sentence containing no “free” variables. That is, all variables are “bound” by universal or existential quantifiers.
 $(\forall x)P(x,y)$ has x bound as a universally quantified variable, but y is free

BNF Grammar for First-Order Logic

- The Backus–Naur Form (BNF)
- Defines the precise syntax for valid sentences, atomic expressions, terms, and quantifiers

```
Sentence      → AtomicSentence | ComplexSentence
AtomicSentence → Predicate | Predicate ( Term, ... ) | Term = Term
ComplexSentence → ( Sentence ) | [ Sentence ] | ¬ Sentence
                | Sentence ∧ Sentence | Sentence ∨ Sentence
                | Sentence ⇒ Sentence | Sentence ⇔ Sentence
                | Quantifier Variable, ... Sentence
Term           → Function ( Term, ... ) | Constant | Variable
Quantifier     → ∀ | ∃
Constant      → A | X1 | John | ...
Variable       → a | x | s | ...
Predicate      → True | False | After | Loves | Raining | ...
Function       → Mother | LeftLeg | ...
```

A BNF for FOL

```
S := <Sentence> ;
<Sentence> := <AtomicSentence> |
    <Sentence> <Connective> <Sentence> |
    <Quantifier> <Variable>, ... <Sentence> |
    "NOT" <Sentence> |
    "(" <Sentence> ")";
<AtomicSentence> := <Predicate> "(" <Term>, ... ")" |
    <Term> "=" <Term>;
<Term> := <Function> "(" <Term>, ... ")" |
    <Constant> |
    <Variable>;
<Connective> := "AND" | "OR" | "IMPLIES" | "EQUIVALENT";
<Quantifier> := "EXISTS" | "FORALL" ;
<Constant> := "A" | "X1" | "John" | ... ;
<Variable> := "a" | "x" | "s" | ... ;
<Predicate> := "Before" | "HasColor" | "Raining" | ... ;
<Function> := "Mother" | "LeftLegOf" | ... ;
```

Quantifiers

- **Universal quantification**

- $(\forall x)P(x)$ means P holds for **all** values of x in domain associated with variable
- E.g., $(\forall x) \text{dolphin}(x) \rightarrow \text{mammal}(x)$

- **Existential quantification**

- $(\exists x)P(x)$ means P holds for **some** value of x in domain associated with variable
- E.g., $(\exists x) \text{mammal}(x) \wedge \text{lays_eggs}(x)$
- Permits one to make a statement about some object without naming it

Quantifiers

- Universal quantifiers often used with *implies to form rules*:
 $(\forall x) \text{ student}(x) \rightarrow \text{smart}(x)$ means “All students are smart”
- Universal quantification is *rarely used to make blanket statements* about every individual in the world:
 $(\forall x) \text{ student}(x) \wedge \text{smart}(x)$ means “Everyone in the world is a student and is smart”
- Existential quantifiers are usually used with “and” to specify a list of properties about an individual:
 $(\exists x) \text{ student}(x) \wedge \text{smart}(x)$ means “There is a student who is smart”
- Common mistake: represent this sentence in FOL as:
 $(\exists x) \text{ student}(x) \rightarrow \text{smart}(x)$
 - What does this sentence mean?

Quantifier Scope

- FOL sentences have structure, like programs
- In particular, the variables in a sentence have a scope
- For example, suppose we want to say
 - “everyone who is alive loves someone”
 - $(\forall x) \text{ alive}(x) \rightarrow (\exists y) \text{ loves}(x,y)$
- Here’s how we scope the variables

$$(\forall x) \text{ alive}(x) \rightarrow (\exists y) \text{ loves}(x,y)$$

 Scope of x
 Scope of y

Quantifier Scope

- **Switching order of universal quantifiers *does not* change the meaning**
 - $(\forall x)(\forall y)P(x,y) \leftrightarrow (\forall y)(\forall x) P(x,y)$
 - “Dogs hate cats”
- **You can switch order of existential quantifiers**
 - $(\exists x)(\exists y)P(x,y) \leftrightarrow (\exists y)(\exists x) P(x,y)$
 - “A cat killed a dog”
- **Switching order of universals and existentials *does* change meaning:**
 - **Everyone likes someone: $(\forall x)(\exists y) \text{ likes}(x,y)$**
 - **Someone is liked by everyone: $(\exists y)(\forall x) \text{ likes}(x,y)$**

In their simplest form, **De Morgan's Laws** can be stated as:

1.The negation of a conjunction is the disjunction of the negations.

- In English: "Not (A and B)" is the same as "(Not A) or (Not B)."
- In logical notation: $\neg(A \wedge B) \equiv (\neg A \vee \neg B)$

2.The negation of a disjunction is the conjunction of the negations.

- In English: "Not (A or B)" is the same as "(Not A) and (Not B)."
- In logical notation: $\neg(A \vee B) \equiv (\neg A \wedge \neg B)$

Connections between All and Exists

- We can relate sentences involving $\forall$ and $\exists$ using **De Morgan's laws**:

$$1. (\forall x) \neg P(x) \leftrightarrow \neg (\exists x) P(x)$$

$$2. \neg (\forall x) P \leftrightarrow (\exists x) \neg P(x)$$

$$3. (\forall x) P(x) \leftrightarrow \neg (\exists x) \neg P(x)$$

$$4. (\exists x) P(x) \leftrightarrow \neg (\forall x) \neg P(x)$$

- Examples

$$1. \text{All dogs don't like cats} \leftrightarrow \text{No dogs like cats}$$

$$2. \text{Not all dogs dance} \leftrightarrow \text{There is a dog that doesn't dance}$$

$$3. \text{All dogs sleep} \leftrightarrow \text{There is no dog that doesn't sleep}$$

$$4. \text{There is a dog that talks} \leftrightarrow \text{Not all dogs can't talk}$$

Quantified inference rules

- Universal instantiation
 - $\forall x P(x) \therefore P(A)$
- Universal generalization
 - $P(A) \wedge P(B) \dots \therefore \forall x P(x)$
- Existential instantiation
 - $\exists x P(x) \therefore P(F)$
- Existential generalization
 - $P(A) \therefore \exists x P(x)$

← **skolem constant F**

F must be a “new” constant not appearing in the KB

Universal instantiation (a.k.a. universal elimination)

- If $(\forall x) P(x)$ is true, then $P(C)$ is true, where C is *any* constant in the domain of x , e.g.:

$$(\forall x) \text{ eats}(\text{John}, x) \Rightarrow \\ \text{eats}(\text{John}, \text{Cheese18})$$

- Note that function applied to ground terms is also a constant

$$(\forall x) \text{ eats}(\text{John}, x) \Rightarrow \\ \text{eats}(\text{John}, \text{contents}(\text{Box42}))$$

Existential instantiation (a.k.a. existential elimination)

- From $(\exists x) P(x)$ infer $P(c)$, e.g.:
 - $(\exists x) \text{ eats}(\text{Mickey}, x) \rightarrow \text{eats}(\text{Mickey}, \text{Stuff345})$
- The variable is replaced by a **brand-new constant** not occurring in this or any sentence in the KB
- Also known as skolemization; constant is a **skolem constant**
- We don't want to accidentally draw other inferences about it by introducing the constant
- Can use this to reason about unknown objects, rather than constantly manipulating existential quantifiers

Existential generalization (a.k.a. existential introduction)

- If $P(c)$ is true, then $(\exists x) P(x)$ is inferred, e.g.:
Eats(Mickey, Cheese18) $\Rightarrow$
 $(\exists x) \text{ eats}(\text{Mickey}, x)$
- All instances of the given constant symbol are replaced by the new variable symbol
- Note that the variable symbol cannot already exist anywhere in the expression

Translating English to FOL

Every gardener likes the sun

$$\forall x \text{ gardener}(x) \rightarrow \text{likes}(x, \text{Sun})$$

You can fool some of the people all of the time

$$\exists x \forall t \text{ person}(x) \wedge \text{time}(t) \rightarrow \text{can-fool}(x, t)$$

You can fool all of the people some of the time

$$\forall x \exists t (\text{person}(x) \rightarrow \text{time}(t) \wedge \text{can-fool}(x, t))$$

$$\forall x (\text{person}(x) \rightarrow \exists t (\text{time}(t) \wedge \text{can-fool}(x, t)))$$

Note 2
possible
readings of
NL sentence

All purple mushrooms are poisonous

$$\forall x (\text{mushroom}(x) \wedge \text{purple}(x)) \rightarrow \text{poisonous}(x)$$

Translating English to FOL

No purple mushroom is poisonous (two ways)

$$\neg \exists x \text{ purple}(x) \wedge \text{mushroom}(x) \wedge \text{poisonous}(x)$$

$$\forall x (\text{mushroom}(x) \wedge \text{purple}(x)) \rightarrow \neg \text{poisonous}(x)$$

There are exactly two purple mushrooms

$$\begin{aligned} &\exists x \exists y \text{ mushroom}(x) \wedge \text{purple}(x) \wedge \text{mushroom}(y) \wedge \\ &\text{purple}(y) \wedge \neg(x=y) \wedge \forall z (\text{mushroom}(z) \wedge \text{purple}(z)) \\ &\rightarrow ((x=z) \vee (y=z)) \end{aligned}$$

Obama is not short

$$\neg \text{short}(\text{Obama})$$

Example: A simple genealogy KB by FOL

- **Build a small genealogy knowledge base using FOL that**
 - contains facts of immediate family relations (spouses, parents, etc.)
 - contains definitions of more complex relations (ancestors, relatives)
 - is able to answer queries about relationships between people
- **Predicates:**
 - parent(x, y), child(x, y), father(x, y), daughter(x, y), etc.
 - spouse(x, y), husband(x, y), wife(x, y)
 - ancestor(x, y), descendant(x, y)
 - male(x), female(y)
 - relative(x, y)
- **Facts:**
 - husband(Joe, Mary), son(Fred, Joe)
 - spouse(John, Nancy), male(John), son(Mark, Nancy)
 - father(Jack, Nancy), daughter(Linda, Jack)
 - daughter(Liz, Linda)

family tree

- **Rules for genealogical relations**

- $(\forall x,y) \text{ parent}(x, y) \leftrightarrow \text{child}(y, x)$
 $(\forall x,y) \text{ father}(x, y) \leftrightarrow \text{parent}(x, y) \wedge \text{male}(x)$;similarly for *mother*(x, y)
 $(\forall x,y) \text{ daughter}(x, y) \leftrightarrow \text{child}(x, y) \wedge \text{female}(x)$;similarly for *son*(x, y)
- $(\forall x,y) \text{ husband}(x, y) \leftrightarrow \text{spouse}(x, y) \wedge \text{male}(x)$;similarly for *wife*(x, y)
 $(\forall x,y) \text{ spouse}(x, y) \leftrightarrow \text{spouse}(y, x)$;**spouse relation is symmetric**
- $(\forall x,y) \text{ parent}(x, y) \rightarrow \text{ancestor}(x, y)$
 $(\forall x,y)(\exists z) \text{ parent}(x, z) \wedge \text{ancestor}(z, y) \rightarrow \text{ancestor}(x, y)$
- $(\forall x,y) \text{ descendant}(x, y) \leftrightarrow \text{ancestor}(y, x)$
- $(\forall x,y)(\exists z) \text{ ancestor}(z, x) \wedge \text{ancestor}(z, y) \rightarrow \text{relative}(x, y)$
;related by common ancestry
 $(\forall x,y) \text{ spouse}(x, y) \rightarrow \text{relative}(x, y)$;related by marriage
 $(\forall x,y)(\exists z) \text{ relative}(z, x) \wedge \text{relative}(z, y) \rightarrow \text{relative}(x, y)$;**transitive**
 $(\forall x,y) \text{ relative}(x, y) \leftrightarrow \text{relative}(y, x)$;**symmetric**

- **Queries**

- $\text{ancestor}(\text{Jack}, \text{Fred})$; the answer is yes
- $\text{relative}(\text{Liz}, \text{Joe})$; the answer is yes
- $\text{relative}(\text{Nancy}, \text{Matthew})$
;no answer in general, no if under closed world assumption
- $(\exists z) \text{ ancestor}(z, \text{Fred}) \wedge \text{ancestor}(z, \text{Liz})$

Semantics of FOL

- **Domain M :** the set of all objects in the world (of interest)
- **Interpretation I :** includes
 - Assign each constant to an object in M
 - Define each function of n arguments as a mapping $M^n \Rightarrow M$
 - Define each predicate of n arguments as a mapping $M^n \Rightarrow \{T, F\}$
 - Therefore, every ground predicate with any instantiation will have a truth value
 - In general there is an infinite number of interpretations because $|M|$ is infinite
- **Define logical connectives:** $\sim, \wedge, \vee, \Rightarrow, \Leftrightarrow$ as in PL
- **Define semantics of $(\forall x)$ and $(\exists x)$**
 - $(\forall x) P(x)$ is true iff $P(x)$ is true under all interpretations
 - $(\exists x) P(x)$ is true iff $P(x)$ is true under some interpretation

- **Model:** an interpretation of a set of sentences such that every sentence is *True*
- **A sentence is**
 - **satisfiable** if it is true under some interpretation
 - **valid** if it is true under all possible interpretations
 - **inconsistent** if there does not exist any interpretation under which the sentence is true
- **Logical consequence:** $S \models X$ if all models of S are also models of X

Axioms, definitions and theorems

- **Axioms** are facts and rules that attempt to capture all of the (important) facts and concepts about a domain; axioms can be used to prove **theorems**
 - Mathematicians don't want any unnecessary (dependent) axioms, i.e. ones that can be derived from other axioms
 - Dependent axioms can make reasoning faster, however
 - Choosing a good set of axioms for a domain is a design problem
- A **definition** of a predicate is of the form " $p(X) \leftrightarrow \dots$ " and can be decomposed into two parts
 - Necessary** description: " $p(x) \rightarrow \dots$ "
 - Sufficient** description " $p(x) \leftarrow \dots$ "
 - Some concepts don't have complete definitions (e.g., $\text{person}(x)$)

More on definitions

Example: define $\text{father}(x, y)$ by $\text{parent}(x, y)$ and $\text{male}(x)$

- **$\text{parent}(x, y)$** is a necessary (but not sufficient) description of $\text{father}(x, y)$

$$\text{father}(x, y) \rightarrow \text{parent}(x, y)$$

- **$\text{parent}(x, y) \wedge \text{male}(x) \wedge \text{age}(x, 35)$** is a sufficient (but not necessary) description of $\text{father}(x, y)$:

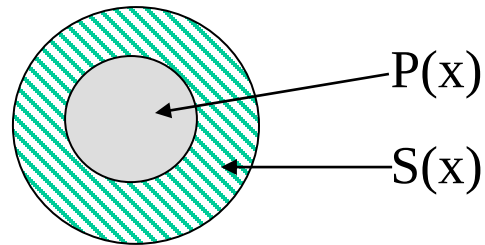
$$\text{father}(x, y) \leftarrow \text{parent}(x, y) \wedge \text{male}(x) \wedge \text{age}(x, 35)$$

- **$\text{parent}(x, y) \wedge \text{male}(x)$** is a necessary and sufficient description of $\text{father}(x, y)$

$$\text{parent}(x, y) \wedge \text{male}(x) \leftrightarrow \text{father}(x, y)$$

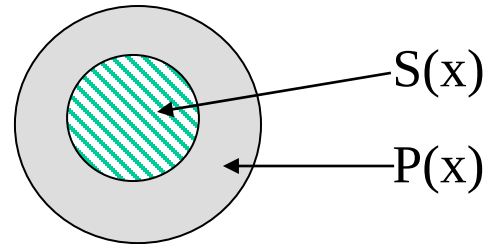
More on definitions

$S(x)$ is a
necessary
condition of $P(x)$



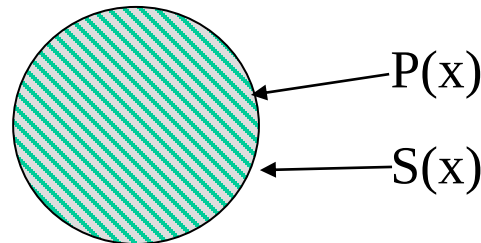
$$(\forall x) P(x) \Rightarrow S(x)$$

$S(x)$ is a
sufficient
condition of $P(x)$



$$(\forall x) P(x) \Leftarrow S(x)$$

$S(x)$ is a
necessary and
sufficient
condition of $P(x)$



$$(\forall x) P(x) \Leftrightarrow S(x)$$

Summary-Recall.....

- First order logic (FOL) introduces predicates, functions and quantifiers
- Much more expressive, but reasoning is more complex
 - Reasoning is semi-decidable
- FOL is a common AI knowledge representation language
- Other KR languages (e.g., OWL) are often defined by mapping them to FOL
- FOL variables range over objects

Inference in First Order Logic

Outline

- Reducing first-order inference to propositional inference
- Unification
- Generalized Modus Ponens
- Forward chaining
- Backward chaining
- Resolution

Inference Rules for FOL

- Inference rules for PL apply to FOL as well (Modus Ponens, And-Introduction, And-Elimination, etc.)
- New (sound) inference rules for use with **quantifiers**:
 - Universal Elimination
 - Existential Introduction
 - Existential Elimination
 - Generalized Modus Ponens (GMP)
- **Resolution**
 - Clause form (CNF in FOL)
 - Unification (consistent variable substitution)
 - Refutation resolution (proof by contradiction)

Universal Elimination $(\forall x) P(x) \vdash P(c)$.

- If $(\forall x) P(x)$ is true, then $P(c)$ is true for any constant c in the domain of x , i.e., $(\forall x) P(x) \models P(c)$.
- Replace all occurrences of x in the scope of $\forall x$ by the **same** ground term (a constant or a ground function).
- Example: $(\forall x) \text{eats}(\text{Ziggy}, x) \vdash \text{eats}(\text{Ziggy}, \text{IceCream})$

Existential Introduction $P(c) \vdash (\exists x) P(x)$

- If $P(c)$ is true, so is $(\exists x) P(x)$, i.e., $P(c) \models (\exists x) P(x)$
- Replace all instances of the given constant symbol by the same **new** variable symbol.
- Example $\text{eats}(\text{Ziggy}, \text{IceCream}) \vdash (\exists x) \text{eats}(\text{Ziggy}, x)$

Existential Elimination

- From $(\exists x) P(x)$ infer $P(c)$, i.e., $(\exists x) P(x) \models P(c)$, where c is a new constant symbol,
 - All we know is there must be some constant that makes this true, so we can introduce a brand new one to stand in for that constant, *even though we don't know exactly what that constant refer to.*
 - Example: $(\exists x) \text{eats}(\text{Ziggy}, x) \models \text{eats}(\text{Ziggy}, \text{Stuff})$

- Things become more complicated when there are universal quantifiers

$$(\forall x)(\exists y) \text{ eats}(x, y) \models (\forall x)\text{eats}(x, \text{Stuff}) ???$$

$$(\forall x)(\exists y) \text{ eats}(x, y) \models \text{eats}(\text{Ziggy}, \text{Stuff}) ???$$

- **Introduce a new function *food_sk(x)* to stand for $\exists y$ because that *y* depends on *x***

$$(\forall x)(\exists y) \text{ eats}(x, y) \dashv\vdash (\forall x)\text{eats}(x, \text{food_sk}(x))$$

$$(\forall x)(\exists y) \text{ eats}(x, y) \dashv\vdash \text{eats}(\text{Ziggy}, \text{food_sk}(\text{Ziggy}))$$

- What exactly the function *food_sk(.)* does is unknown, except that it takes *x* as its argument

- **The process of existential elimination is called “*Skolemization*”, and the new, unique constants (e.g., *Stuff*) and functions (e.g., *food_sk(.)*) are called **Skolem constants** and **Skolem functions**.**

Generalized Modus Ponens (GMP)

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q\theta} \quad \text{where } p_i'\theta = p_i \theta \text{ for all } i$$

p_1' is *King(John)*

p_1 is *King(x)*

p_2' is *Greedy(y)*

p_2 is *Greedy(x)*

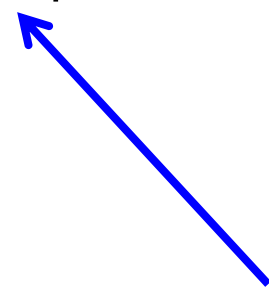
θ is $\{x/\text{John}, y/\text{John}\}$

q is *Evil(x)*

$q\theta$ is *Evil(John)*

- GMP used with KB of definite clauses (**exactly** one positive literal)
- All variables assumed universally quantified
-

Horn Clause



Generalized Modus Ponens (GMP)

- Combines And-Introduction, Universal-Elimination, and Modus Ponens

- Ex: $P(c), Q(c), (\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x) \vdash R(c)$

$P(c), Q(c) \vdash P(c) \wedge Q(c)$ *(by and-introduction)*

$(\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x)$

$\vdash (P(c) \wedge Q(c)) \Rightarrow R(c)$ *(by universal-elimination)*

$P(c) \wedge Q(c), (P(c) \wedge Q(c)) \Rightarrow R(c) \vdash R(c)$ *(by modus ponens)*

- **All occurrences of a quantified variable must be instantiated to the same constant.**

$P(a), Q(c), (\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x) \vdash R(c)$

because all occurrences of x must either be instantiated to a or c which makes the modus ponens rule not applicable.

Resolution for FOL

- Resolution rule operates on two *clauses*
 - A clause is a **disjunction** of literals (without explicit quantifiers)
 - Relationship between clauses in KB is **conjunction**
- Resolution Rule for FOL:
 - clause C1: $(l_1, l_2, \dots l_i, \dots l_n)$ and
clause C2: $(l'_1, l'_2, \dots l'_j, \dots l'_m)$
 - if l_i and l'_j are two **opposite literals** (e.g., P and $\sim P$) and their argument lists can be made the same (**unified**) by a set of variable bindings $\theta = \{x_1/y_1, \dots X_k/y_k\}$ where $x_1, \dots X_k$ are variables and $y_1, \dots Y_k$ are terms, then derive a new clause (called resolvent)
$$\text{subst}((l_1, l_2, \dots l_n, l'_1, l'_2, \dots l'_m), \theta)$$

where function $\text{subst}(\text{expression}, \theta)$ returns a new expression by applying all variable bindings in θ to the original expression

We need answers to the following questions

- How to convert FOL sentences to clause form (especially how to remove quantifiers)
 - How to unify two argument lists, i.e., how to find their most general unifier (**mgu**) θ
 - How to determine which two clauses in KB should be resolved next (among all resolvable pairs of clauses) and how to determine a proof is completed
-
- **The most general unifier (MGU)**

Converting FOL sentences to clause form

- **Clauses are quantifier free CNF of FOL sentences**
- Basic ideas
 - How to handle quantifiers
 - Careful on quantifiers with preceding negations (explicit or implicit)
 $\sim \forall x P(x)$ is really $\exists x \sim P(x)$
 $(\forall x P(x)) \Rightarrow (\forall y Q(y)) \equiv \sim(\forall x P(x)) \vee (\forall y Q(y))$
 $\equiv \exists x \sim P(x) \vee \forall y Q(y)$
 - **Eliminate true existential quantifier by Skolemization**
 - For true universally quantified variables, treat them as such without quantifiers
 - How to convert to CNF (similar to PL but need to work with quantifiers)

Conversion procedure

step 1: remove all “ $\Rightarrow$ ” and “ $\Leftrightarrow$ ” operators

(using $P \Rightarrow Q \equiv \sim P \vee Q$ and $P \Leftrightarrow Q \equiv P \Rightarrow Q \wedge Q \Rightarrow P$)

step 2: move all negation signs to individual predicates

(using de Morgan's law)

step 3: remove all existential quantifiers $\exists y$

case 1: y is not in the scope of any universally quantified variable,
then replace all occurrences of y by a skolem constant

case 2: if y is in scope of universally quantified variables $x_1, \dots, x_i$,
then replace all occurrences of y by a skolem function

step 4: remove all universal quantifiers $\forall x$ (with the understanding that all remaining variables are universally quantified)

step 5: convert the sentence into CNF (using distribution law, etc)

step 6: use parenthesis to separate all disjunctions, then drop all $\vee$'s
and $\wedge$'s

Conversion examples

$\forall x (P(x) \wedge Q(x) \Rightarrow R(x))$

$\forall x \sim(P(x) \wedge Q(x)) \vee R(x)$ (by step 1)

$\forall x \sim P(x) \vee \sim Q(x) \vee R(x)$ (by step 2)

$\sim P(x) \vee \sim Q(x) \vee R(x)$ (by step 4)

$(\sim P(x), \sim Q(x), R(x))$ (by step 6)

$\exists y \text{ rose}(y) \wedge \text{yellow}(y)$

$\text{rose}(c) \wedge \text{yellow}(c)$

(where **c is a skolem constant**)

$(\text{rose}(c)), (\text{yellow}(c))$

$\forall x [\text{person}(x) \Rightarrow \exists y (\text{person}(y) \wedge \text{father}(y, x))]$

$\forall x [\sim \text{person}(x) \vee \exists y (\text{person}(y) \wedge \text{father}(y, x))]$ (by step 1)

$\forall x [\sim \text{person}(x) \vee (\text{person}(f_sk(x)) \wedge \text{father}(f_sk(x), x))]$ (by step 3)

$\sim \text{person}(x) \vee (\text{person}(f_sk(x)) \wedge \text{father}(f_sk(x), x))$ (by step 4)

$(\sim \text{person}(x) \vee \text{person}(f_sk(x))) \wedge (\sim \text{person}(x) \vee \text{father}(f_sk(x), x))$ (by step 5)

$(\sim \text{person}(x), \text{person}(f_sk(x)), (\sim \text{person}(x), \text{father}(f_sk(x), x)))$ (by step 6)

(where $f_sk(.)$ is a skolem function)

Unification of two clauses

- Basic idea: $\forall x P(x) \Rightarrow Q(x), P(a) \vdash Q(a)$

$(\sim P(x), Q(x)), (P(a))$



$\{x/a\}$ a substitution in which variable x is bound to a

$Q(a)$

- The goal is to find a set of variable bindings so that the argument lists of two opposite literals (in two clauses) can be made the same.
- Only variables can be bound to other things.
 - a and b cannot be unified (different constants in general refer to different objects)
 - a and $f(x)$ cannot be unified (unless the inverse function of f is known, which is not the case for general functions in FOL)
 - $f(x)$ and $g(y)$ cannot be unified (function symbols f and g in general refer to different functions and their exact definitions are different in different interpretations)

- Cannot bind variable x to y if x appears anywhere in y
 - Try to unify x and $f(x)$. If we bind x to $f(x)$ and apply the binding to both x and $f(x)$, we get $f(x)$ and $f(f(x))$ which are still not the same (and will never be made the same no matter how many times the binding is applied)
- Otherwise, bind variable x to y , written as x/y (this guarantees to find the most general unifier, or **mgu**)
 - Suppose both x and y are variables, then they can be made the same by binding both of them to any constant c or any function $f(\cdot)$. Such bindings are less general and impose unnecessary restriction on x and y .
- To unify two terms of the same function symbol, unify their argument lists (**unification is recursive**)
Ex: to unify $f(x)$ and $f(g(b))$, we need to unify x and $g(b)$

- When the argument lists contain multiple terms, unify each pair of terms

Ex. To unify $(x, f(x), \dots)$ $(a, y, \dots)$

1. unify x and a ($\theta = \{x/a\}$)
2. apply θ to the remaining terms in both lists, resulting $(f(a), \dots)$ and $(y, \dots)$
 1. unify $f(a)$ and y with binding $y/f(a)$
 2. apply the new binding $y/f(a)$ to θ
 3. add $y/f(a)$ to new θ

Unification Examples

- $\text{parents}(x, \text{father}(x), \text{mother}(\text{Bill}))$ and $\text{parents}(\text{Bill}, \text{father}(\text{Bill}), y)$
 - unify x and Bill : $\theta = \{x/\text{Bill}\}$
 - unify $\text{father}(\text{Bill})$ and $\text{father}(\text{Bill})$: $\theta = \{x/\text{Bill}\}$
 - unify $\text{mother}(\text{Bill})$ and y : $\theta = \{x/\text{Bill}\}, / \text{mother}(\text{Bill})\}$
- $\text{parents}(x, \text{father}(x), \text{mother}(\text{Bill}))$ and $\text{parents}(\text{Bill}, \text{father}(y), z)$
 - unify x and Bill : $\theta = \{x/\text{Bill}\}$
 - unify $\text{father}(\text{Bill})$ and $\text{father}(y)$: $\theta = \{x/\text{Bill}, y/\text{Bill}\}$
 - unify $\text{mother}(\text{Bill})$ and z : $\theta = \{x/\text{Bill}, y/\text{Bill}, z/\text{mother}(\text{Bill})\}$
- $\text{parents}(x, \text{father}(x), \text{mother}(\text{Jane}))$ and $\text{parents}(\text{Bill}, \text{father}(y), \text{mother}(y))$
 - unify x and Bill : $\theta = \{x/\text{Bill}\}$
 - unify $\text{father}(\text{Bill})$ and $\text{father}(y)$: $\theta = \{x/\text{Bill}, y/\text{Bill}\}$
 - unify $\text{mother}(\text{Jane})$ and $\text{mother}(\text{Bill})$: Failure because Jane and Bill are different constants

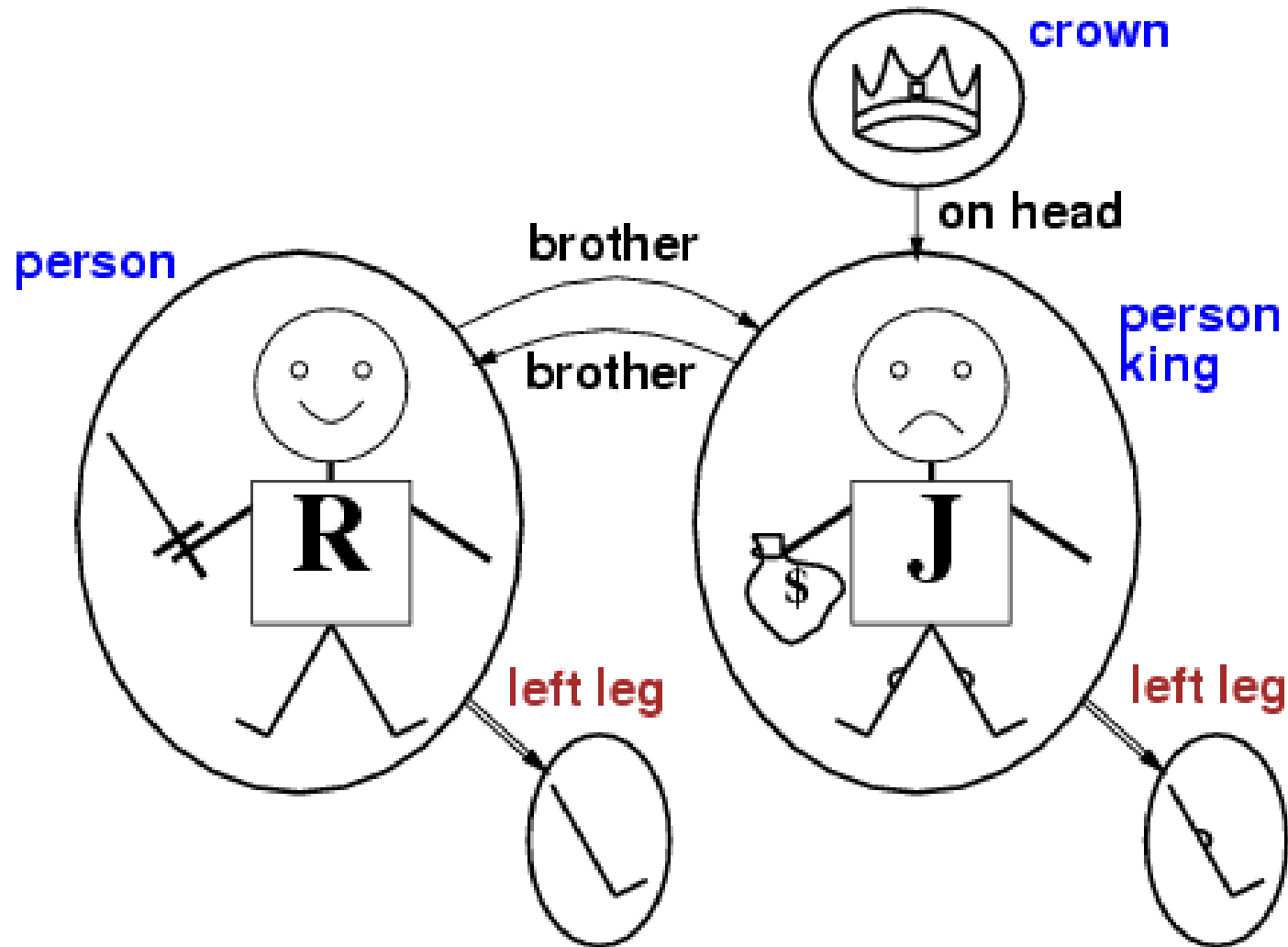
More Unification Examples

- $P(x, g(x), h(b))$ and $P(f(u, a), v, u)$
 - unify x and $f(u, a)$: $\theta = \{x/f(u, a)\}$;
remaining lists: $(g(f(u, a)), h(b))$ and (v, u)
 - unify $g(f(u, a))$ and v : $\theta = \{x/f(u, a), v/g(f(u, a))\}$;
remaining lists: $(h(b))$ and (u)
 - unify $h(b)$ and u : $\theta = \{x/f(h(b), a), v/g(f(h(b), a)), u/h(b)\}$;
- $P(f(x, a), g(x, b))$ and $P(y, g(y, b))$
 - unify $f(x, a)$ and y : $\theta = \{y/f(x, a)\}$
remaining lists: $(g(x, b))$ and $(g(f(x, a), b))$
 - unify x and $f(x, a)$: failure because x is in $f(x, a)$

Unification Algorithm

```
procedure unify(p, q,  $\theta$ )      /* p and q are two lists of terms and  $|p| = |q|$  */  
  if p = empty then return  $\theta$ ; /* success */  
  let r = first(p) and s = first(q);  
  if r = s then return unify(rest(p), rest(q),  $\theta$ );  
  if r is a variable then temp = unify-var(r, s);  
  else if s is a variable then temp = unify-var(s, r);  
    else if both r and s are functions of the same function name then  
      temp = unify(arglist(r), arglist(s), empty);  
      else return “failure”;  
  if temp = “failure” then return “failure”; /* p and q are not unifiable */  
  else  $\theta$  = subst( $\theta$ , temp)  $\cup$  temp; /* apply tmp to old  $\theta$  then insert it into  $\theta$  */  
  return unify(subst(rest(p), tmp), subst(rest(q), tmp),  $\theta$ );  
end{unify}  
procedure unify-var(x, y)  
  if x appears anywhere in y then return “failure”;  
  else return (x/y)  
end{unify-var}
```

Models for FOL: Example



Universal instantiation (UI)

- Every instantiation of a universally quantified sentence is entailed by it:

-

$$\frac{\forall v \alpha}{\text{Subst}(\{v/g\}, \alpha)}$$

for any variable v and ground term g

- E.g., $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ yields:

-

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$

.

.

Existential instantiation (EI)

- For any sentence α , variable v , and constant symbol k that does **not** appear elsewhere in the knowledge base:

-

$$\frac{\exists v \alpha}{\text{Subst}(\{v/k\}, \alpha)}$$

- E.g., $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ yields:

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

provided C_1 is a new constant symbol, called a **Skolem constant**

Reduction to propositional inference

Suppose the KB contains just the following:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

- Instantiating the universal sentence in **all possible** ways, we have:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

- The new KB is **propositionalized**: proposition symbols are

-

$\text{King}(\text{John}), \text{Greedy}(\text{John}), \text{Evil}(\text{John}), \text{King}(\text{Richard}), \text{etc.}$

Reduction contd.

- Every FOL KB can be propositionalized so as to preserve entailment
-
- (A ground sentence is entailed by new KB iff entailed by original KB)
-
- Idea: propositionalize KB and query, apply resolution, return result
-
- Problem: with function symbols, there are infinitely many ground terms,
 - e.g., *Father(Father(Father(John)))*

Reduction contd.

Theorem: Herbrand (1930). If a sentence α is entailed by an FOL KB, it is entailed by a **finite** subset of the propositionalized KB

Idea: For $n = 0$ to ∞ do
 create a propositional KB by instantiating with depth- n terms
 see if α is entailed by this KB

Problem: works if α is entailed, loops if α is not entailed

Theorem: Turing (1936), Church (1936) Entailment for FOL is **semidecidable**

(algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every nonentailed sentence.)

Problems with propositionalization

- Propositionalization seems to generate lots of irrelevant sentences.
- E.g., from:
 -
 - $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
King(John)
 $\forall y \text{ Greedy}(y)$
Brother(Richard, John)
- it seems obvious that *Evil(John)*, but propositionalization produces lots of facts such as *Greedy(Richard)* that are irrelevant
 -
- With p k -ary predicates and n constants, there are $p \cdot n^k$ instantiations.

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$
-

$\theta = \{x/John, y/John\}$ works

• $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$		
•		
p	q	θ
Knows(John,x)	Knows(John,Jane)	
Knows(John,x)	Knows(y,OJ)	
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$
-

$\theta = \{x/John, y/John\}$ works

• $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$		
•		
p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, OJ)$	
$Knows(John, x)$	$Knows(y, Mother(y))$	
$Knows(John, x)$	$Knows(x, OJ)$	

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

•

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

•

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

- **Standardizing apart** eliminates overlap of variables, e.g.,
Knows(z_{17} , OJ)

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$
-

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$
-

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	$\{y/John, x/Mother(John)\}$
Knows(John,x)	Knows(x,OJ)	

- **Standardizing apart** eliminates overlap of variables, e.g.,
Knows(z_{17} , OJ)

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$
-

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$
-

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	$\{y/John, x/Mother(John)\}$
Knows(John,x)	Knows(x,OJ)	$\{fail\}$

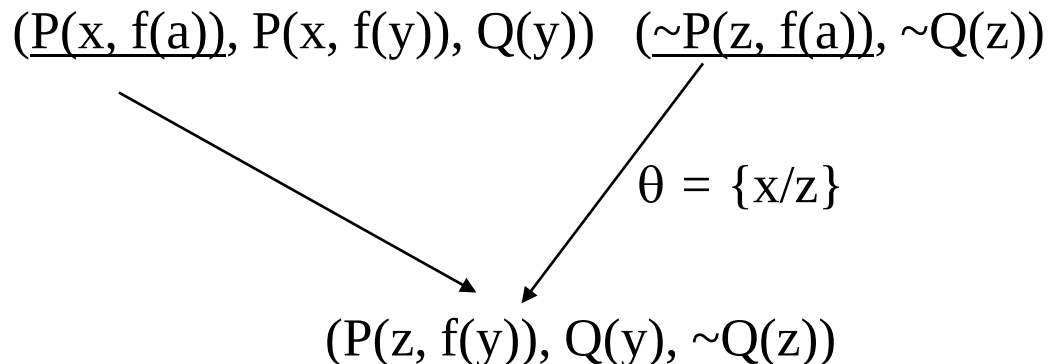
- **Standardizing apart** eliminates overlap of variables, e.g.,
 $Knows(z_{17}, OJ)$

Unification

- To unify $Knows(John, x)$ and $Knows(y, z)$,
- $\theta = \{y/John, x/z\}$ or $\theta = \{y/John, x/John, z/John\}$
- The first unifier is **more general** than the second.
-
- There is a single **most general unifier** (MGU) that is unique up to renaming of variables.
- MGU = $\{y/John, x/z\}$

Resolution in FOL

- Convert all sentences in KB (axioms, definitions, and known facts) and the goal sentence (the theorem to be proved) to clause form
- Two clauses $C1$ and $C2$ can be resolved if and only if r in $C1$ and s in $C2$ are two opposite literals, and their argument list $arglist_r$ and $arglist_s$ are unifiable with $mgu = \theta$.
- Then derive the resolvent sentence: $subst((C1 - \{r\}, C2 - \{s\}), \theta)$
(substitution is applied to all literals in $C1$ and $C2$, but not to any other clauses)
- Example

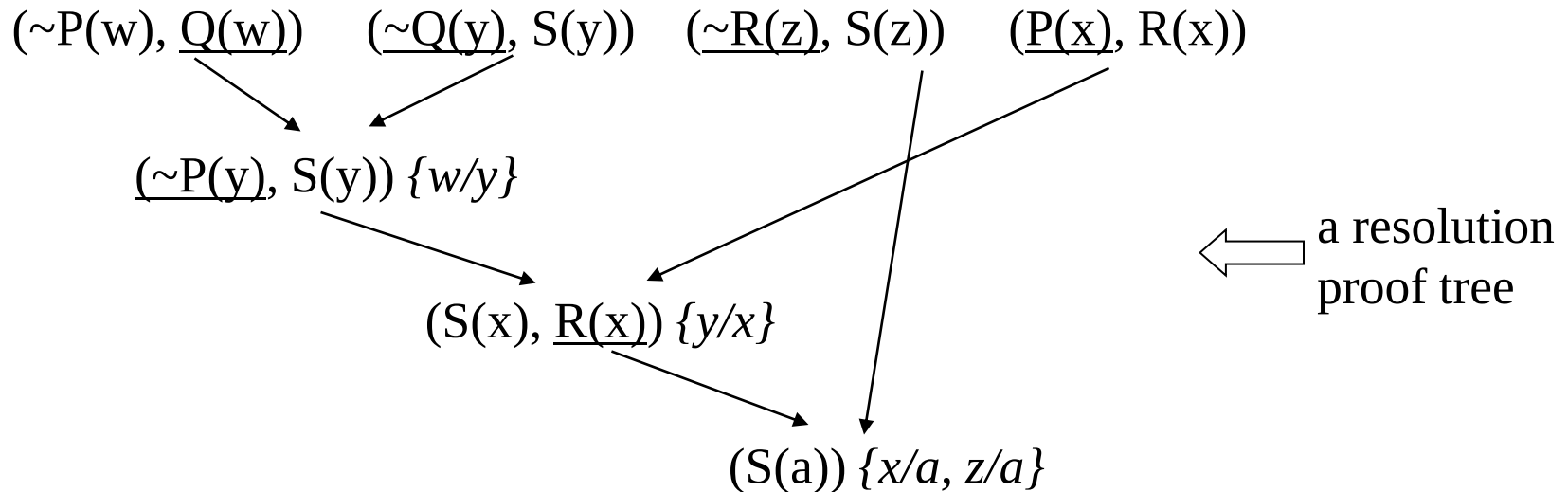


Resolution example

- Prove that

$\forall w P(w) \Rightarrow Q(w), \forall y Q(y) \Rightarrow S(y), \forall z R(z) \Rightarrow S(z), \forall x P(x) \vee R(x) \models \exists u S(u)$

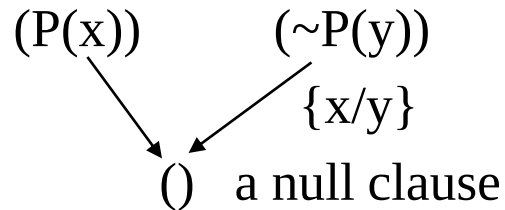
- Convert these sentences to clauses ($\exists u S(u)$ skolemized to $S(a)$)
- Apply resolution



- Problems
 - The theorem $S(a)$ does not actively participate in the proof
 - Hard to determine if a proof (with consistent variable bindings) is completed if the theorem consists of more than one clause

Resolution Refutation: a better proof strategy

- Given a consistent set of axioms KB and goal sentence Q, show that $KB \models Q$.
- **Proof by contradiction:** Add $\sim Q$ to KB and try to prove false.
because $(KB \models Q) \iff (KB \wedge \sim Q \models \text{False}, \text{ or } KB \wedge \sim Q \text{ is inconsistent})$
- How to represent “**false**” in clause form
 - $P(x) \wedge \sim P(y)$ is inconsistent
 - Convert them to clause form then apply resolution

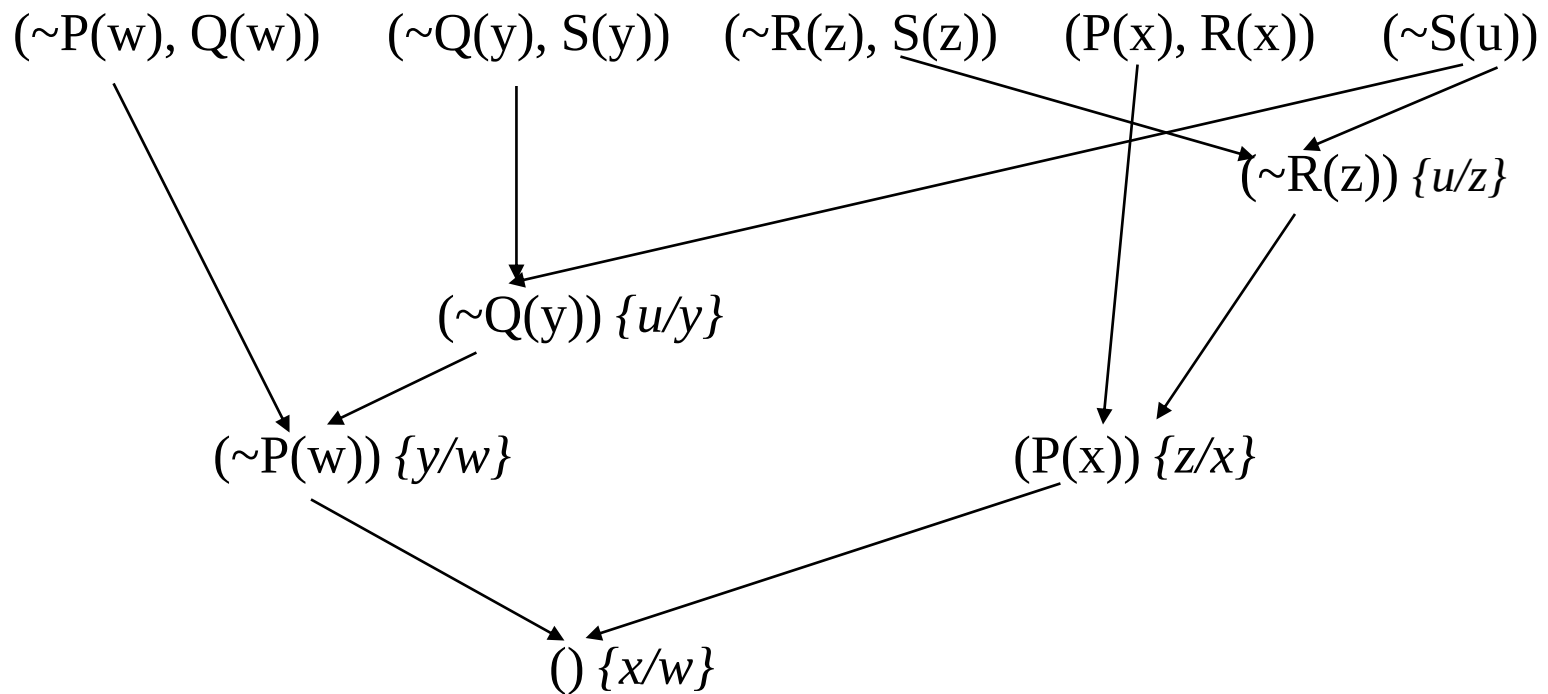


- A null clause represents false (inconsistence/contradiction)
- $KB \models Q$ if we can derive a null clause from $KB \wedge \sim Q$ by resolution

- Prove by resolution refutation that

$$\forall w P(w) \Rightarrow Q(w), \forall y Q(y) \Rightarrow S(y), \forall z R(z) \Rightarrow S(z), \forall x P(x) \vee R(x) \models \exists u S(u)$$

- Convert these sentences to clauses ($\sim \exists u S(u)$ becomes $\sim S(u)$)



Refutation Resolution Procedure

procedure resolution(KB, Q)

/* KB is a set of consistent, true FOL sentences, Q is a goal sentence.

It returns success if $KB \vdash Q$, and failure otherwise */

KB = clause(union(KB, { $\sim Q$ })) /* convert KB and $\sim Q$ to clause form */

while null clause is not in KB **do**

 pick 2 sentences, S1 and S2, in KB that contain a pair of opposite
 literals whose argument lists are unifiable

if none can be found **then return** "failure"

 resolvent = resolution-rule(S1, S2)

 KB = union(KB, {resolvent})

return "success "

end{resolution}

Example knowledge base

- The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.
- Prove that Col. West is a criminal

Example knowledge base contd.

... it is a crime for an American to sell weapons to hostile nations:

$American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Nono ... has some missiles, i.e., $\exists x \text{ Owns}(\text{Nono},x) \wedge \text{Missile}(x)$:

$\text{Owns}(\text{Nono},M_1) \text{ and } \text{Missile}(M_1)$

... all of its missiles were sold to it by Colonel West

$\text{Missile}(x) \wedge \text{Owns}(\text{Nono},x) \Rightarrow \text{Sells}(\text{West},x,\text{Nono})$

Missiles are weapons:

$\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

An enemy of America counts as "hostile":

$\text{Enemy}(x,\text{America}) \Rightarrow \text{Hostile}(x)$

West, who is American ...

$American(\text{West})$

The country Nono, an enemy of America ...

$\text{Enemy}(\text{Nono},\text{America})$

Forward chaining algorithm

```
function FOL-FC-ASK( $KB, \alpha$ ) returns a substitution or false
  repeat until new is empty
     $new \leftarrow \{\}$ 
    for each sentence  $r$  in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$ 
      for each  $\theta$  such that  $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  is not a renaming of a sentence already in  $KB$  or new then do
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not fail then return  $\phi$ 
    add new to  $KB$ 
  return false
```

Forward chaining proof

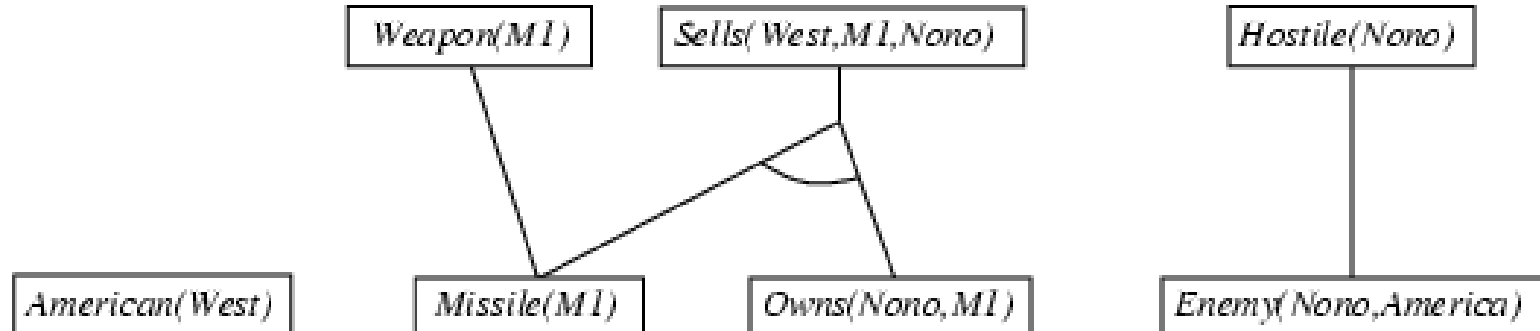
American(West)

Missile(M1)

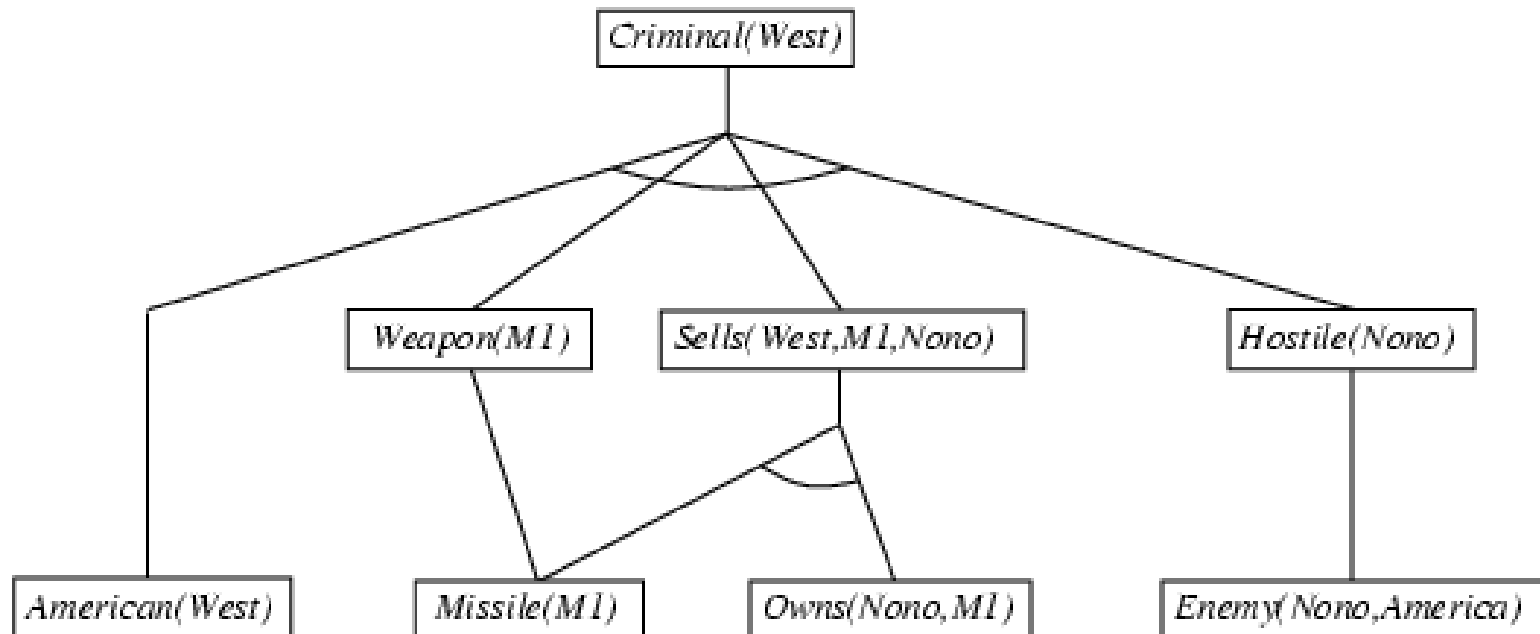
Owns(Nono,M1)

Enemy(Nono,America)

Forward chaining proof



Forward chaining proof



Properties of forward chaining

- Sound and complete for first-order definite clauses
- **Datalog** = first-order definite clauses + **no functions**
- FC terminates for Datalog in a finite number of iterations
- May not terminate in general if α is not entailed
- This is unavoidable: entailment with definite clauses is semidecidable

Efficiency of forward chaining

Incremental forward chaining: no need to match a rule on iteration k if a premise wasn't added on iteration $k-1$
⇒ match each rule whose premise contains a newly added positive literal

Matching itself can be expensive:

Database indexing allows $O(1)$ retrieval of known facts

- e.g., query $Missile(x)$ retrieves $Missile(M_1)$

Forward chaining is widely used in deductive databases

Backward chaining algorithm

```
function FOL-BC-ASK( $KB$ ,  $goals$ ,  $\theta$ ) returns a set of substitutions
  inputs:  $KB$ , a knowledge base
            $goals$ , a list of conjuncts forming a query
            $\theta$ , the current substitution, initially the empty substitution  $\{ \}$ 
  local variables:  $ans$ , a set of substitutions, initially empty

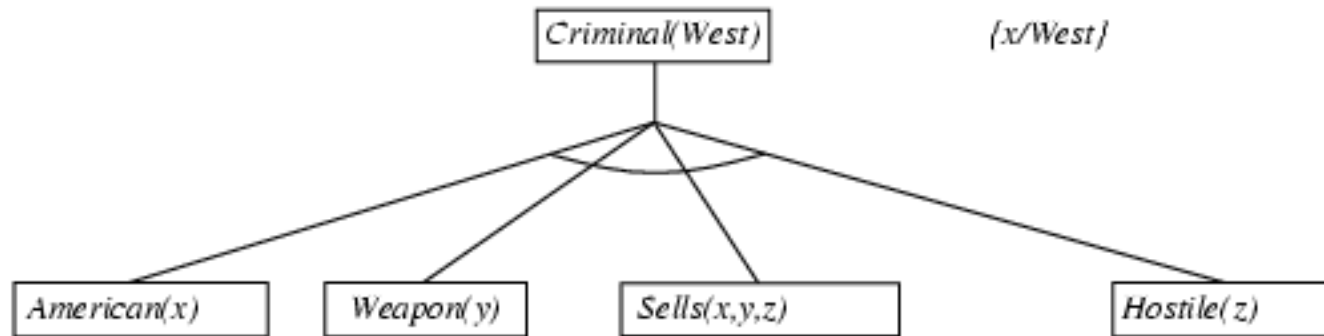
  if  $goals$  is empty then return  $\{ \theta \}$ 
   $q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(goals))$ 
  for each  $r$  in  $KB$  where  $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \dots \wedge p_n \Rightarrow q)$ 
    and  $\theta' \leftarrow \text{UNIFY}(q, q')$  succeeds
       $ans \leftarrow \text{FOL-BC-ASK}(KB, [p_1, \dots, p_n | \text{REST}(goals)], \text{COMPOSE}(\theta, \theta')) \cup ans$ 
  return  $ans$ 
```

$$\text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), p) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, p))$$

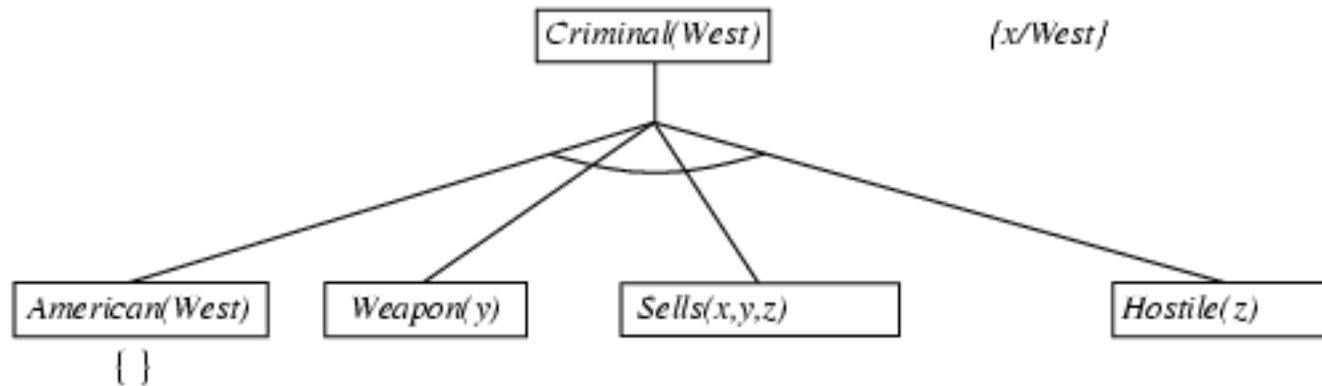
Backward chaining example

Criminal(West)

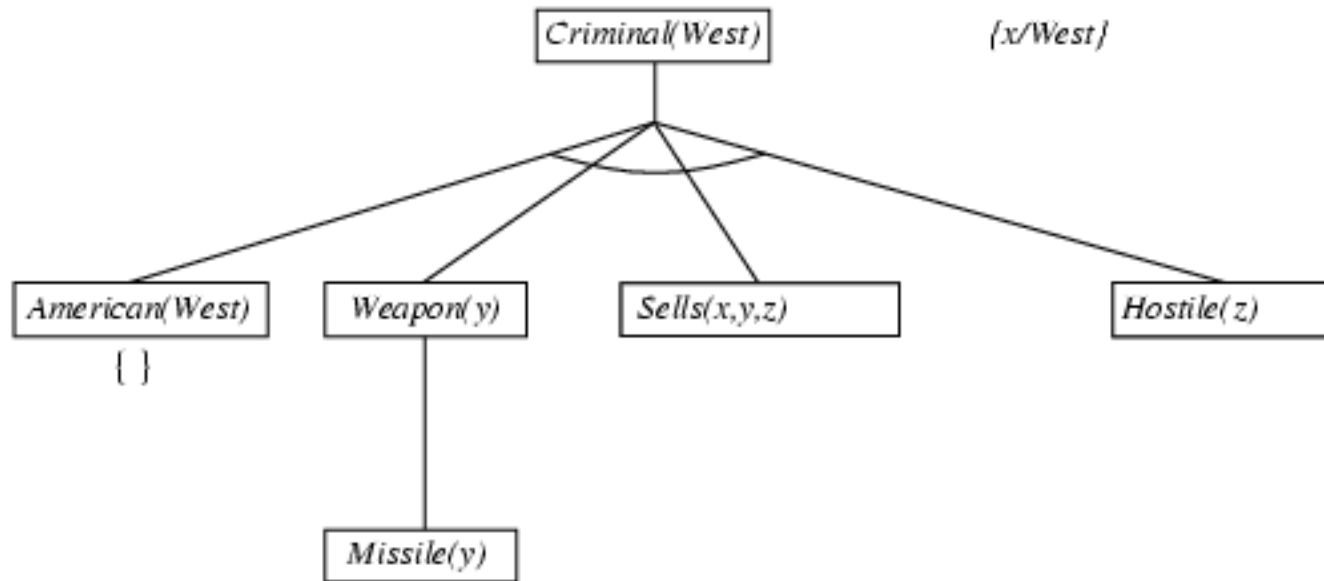
Backward chaining example



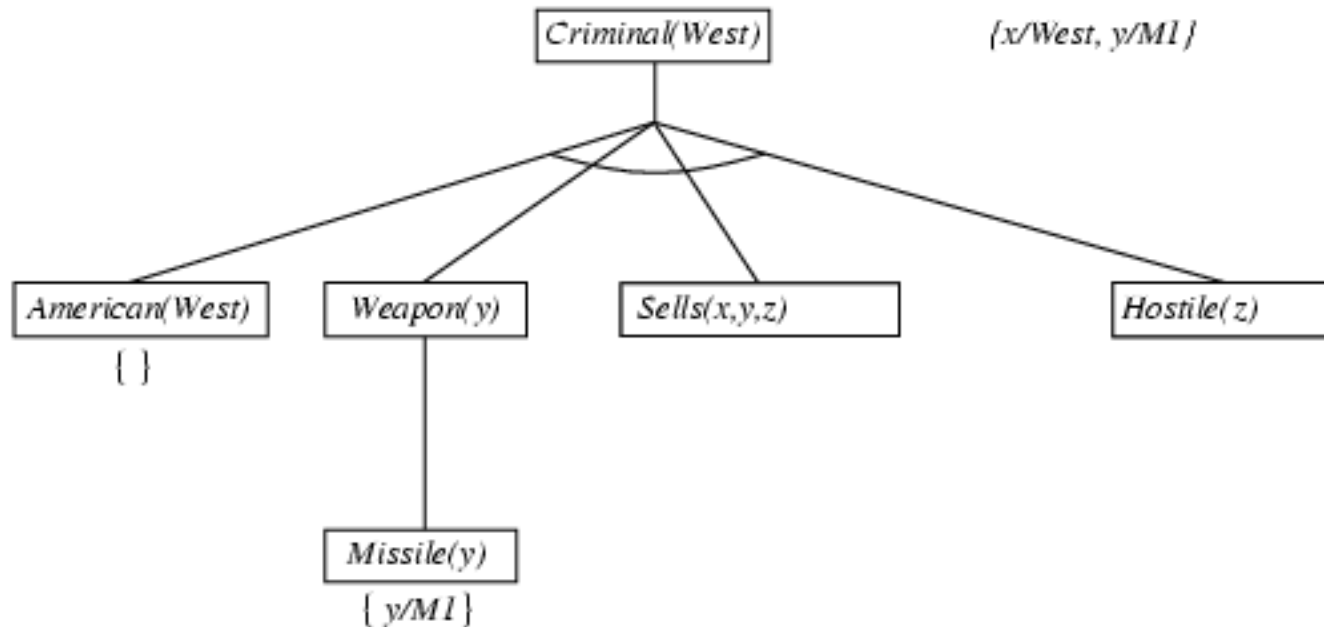
Backward chaining example



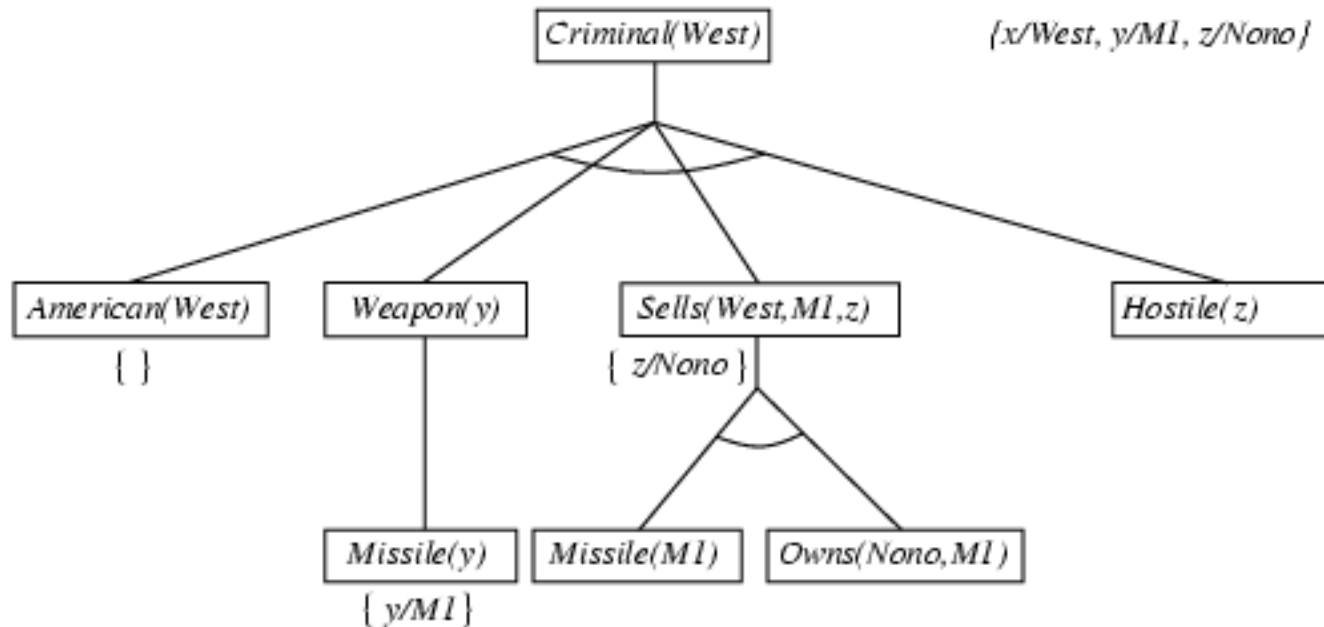
Backward chaining example



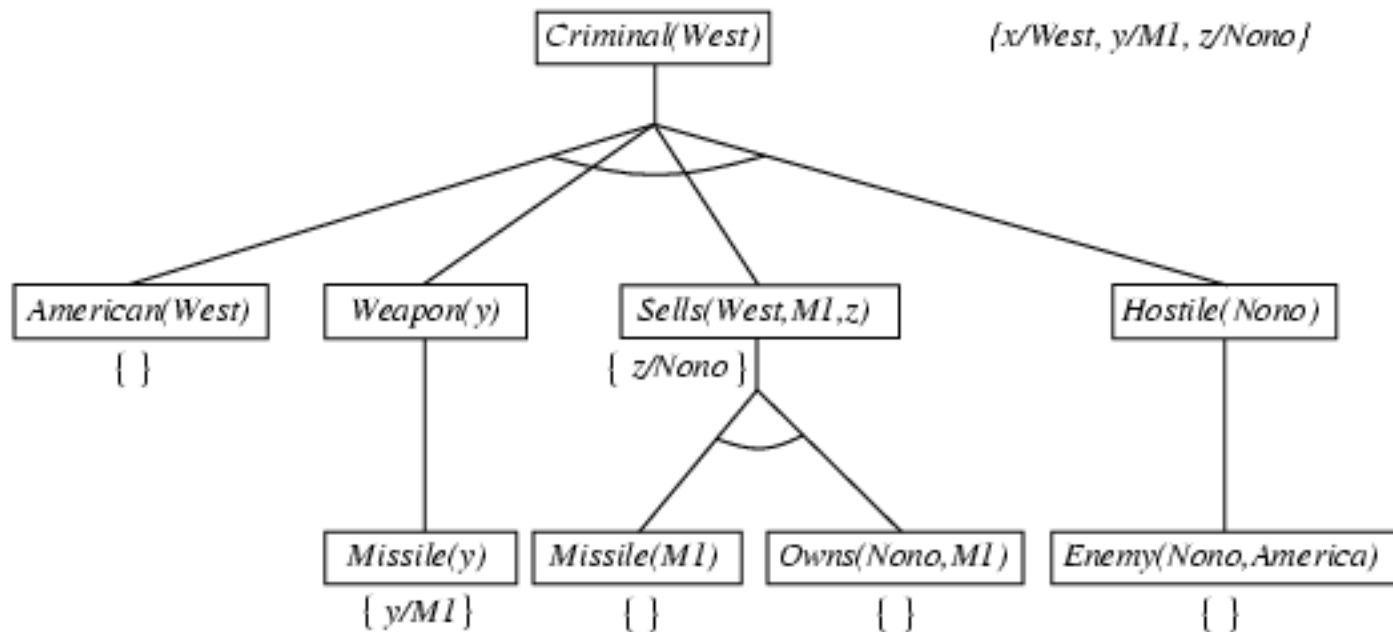
Backward chaining example



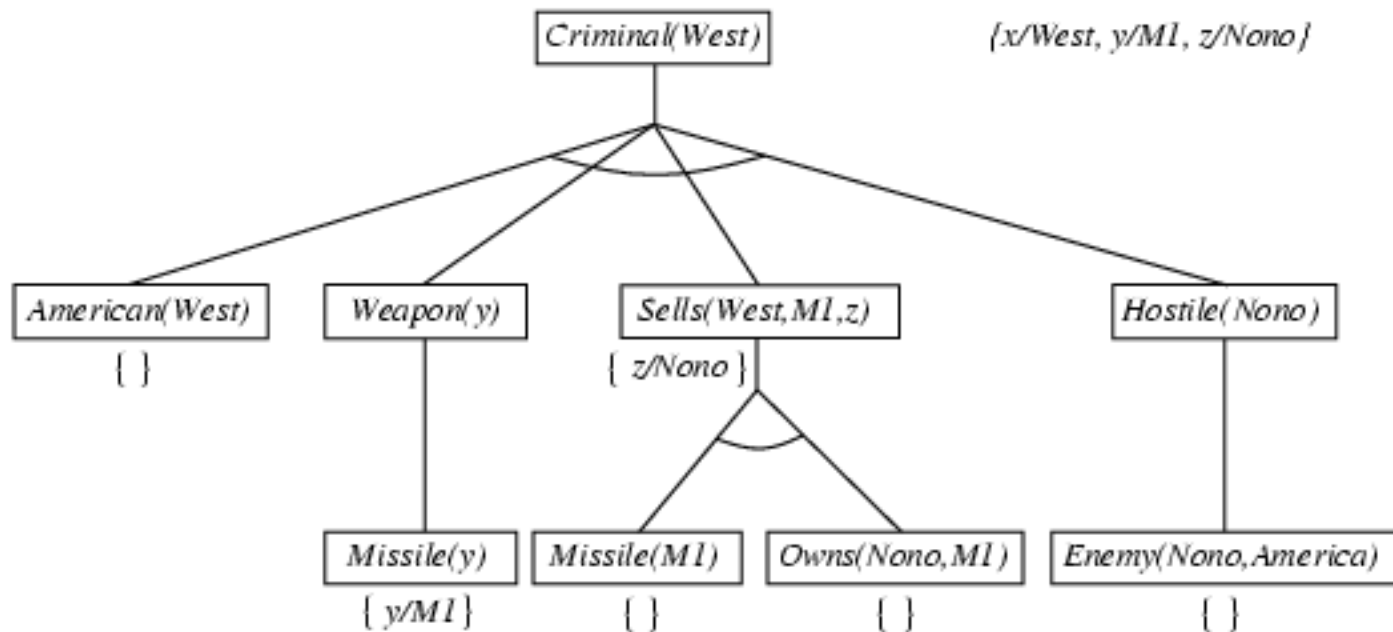
Backward chaining example



Backward chaining example



Backward chaining example



Properties of backward chaining

- Depth-first recursive proof search: space is linear in size of proof
-
- Incomplete due to infinite loops
- - $\Rightarrow$ fix by checking current goal against every goal on stack
- Inefficient due to repeated subgoals (both success and failure)
 - $\Rightarrow$ fix using caching of previous results (extra space)
- Widely used for **logic programming**

Logic programming: Prolog

- Algorithm = Logic + Control
-
- Basis: backward chaining with Horn clauses + bells & whistles
Widely used in Europe, Japan (basis of 5th Generation project)
Compilation techniques $\Rightarrow$ 60 million LIPS
- Program = set of clauses = `head :- literal1, ... literaln.`
- `criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).`
-
- Depth-first, left-to-right backward chaining
- Built-in predicates for arithmetic etc., e.g., `X is Y*Z+3`
- Built-in predicates that have side effects (e.g., input and output)
-
- predicates, assert/retract predicates)
- Closed-world assumption ("negation as failure")
 - e.g., given `alive(X) :- not dead(X).`
 - `alive(joe)` succeeds if `dead(joe)` fails

Prolog

- Appending two lists to produce a third:

-

```
append([], Y, Y) .
```

```
append([X|L], Y, [X|Z]) :- append(L, Y, Z) .
```

- **query:** `append(A, B, [1, 2]) ?`

-

- **answers:** `A=[]        B=[1, 2]`

-

```
A=[1]        B=[2]
```

```
A=[1, 2]    B=[]
```

Resolution: brief summary

- Full first-order version:
-

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{(\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)\theta}$$

where $\text{Unify}(\ell_i, \neg m_j) = \theta$.

- The two clauses are assumed to be standardized apart so that they share no variables.
- For example,

$$\frac{\neg \text{Rich}(x) \vee \text{Unhappy}(x)}{\text{Rich}(\text{Ken}) \text{Unhappy}(\text{Ken})}$$

with $\theta = \{x/\text{Ken}\}$

- Apply resolution steps to $\text{CNF}(\text{KB} \wedge \neg \alpha)$; complete for FOL

Conversion to CNF

- Everyone who loves all animals is loved by someone:

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x,y)] \Rightarrow [\exists y \text{ Loves}(y,x)]$$

- 1. Eliminate biconditionals and implications
- $\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$
- 2. Move $\neg$ inwards: $\neg \forall x p \equiv \exists x \neg p$, $\neg \exists x p \equiv \forall x \neg p$
-

$$\forall x [\exists y \neg(\neg \text{Animal}(y) \vee \text{Loves}(x,y))] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

Conversion to CNF contd.

3. Standardize variables: each quantifier should use a different one

$$\forall x [\exists y \textit{Animal}(y) \wedge \neg \textit{Loves}(x,y)] \vee [\exists z \textit{Loves}(z,x)]$$

4. Skolemize: a more general form of existential instantiation.
Each existential variable is replaced by a **Skolem function** of the enclosing universally quantified variables:

$$\forall x [\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x,F(x))] \vee \textit{Loves}(G(x),x)$$

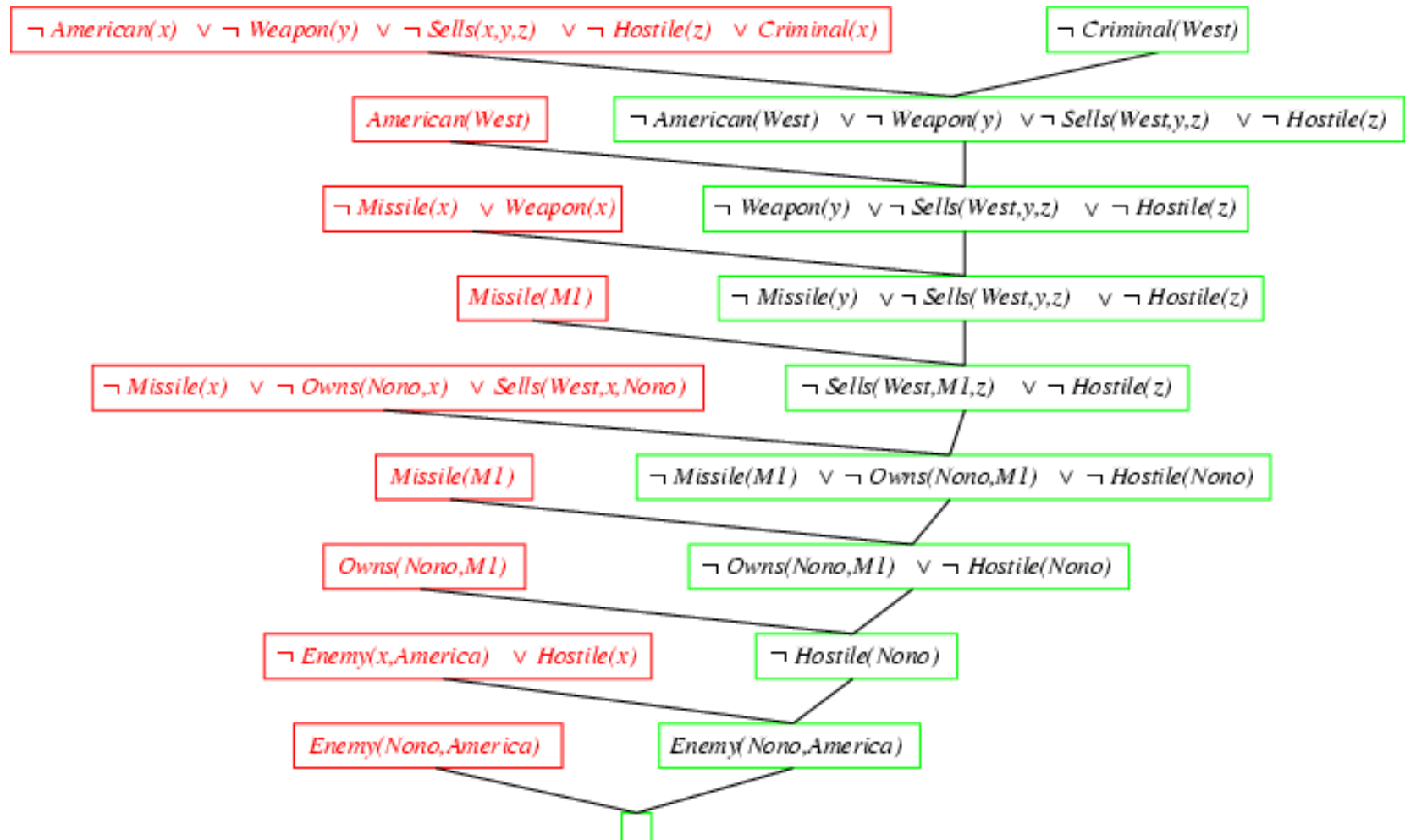
5. Drop universal quantifiers:

$$[\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x,F(x))] \vee \textit{Loves}(G(x),x)$$

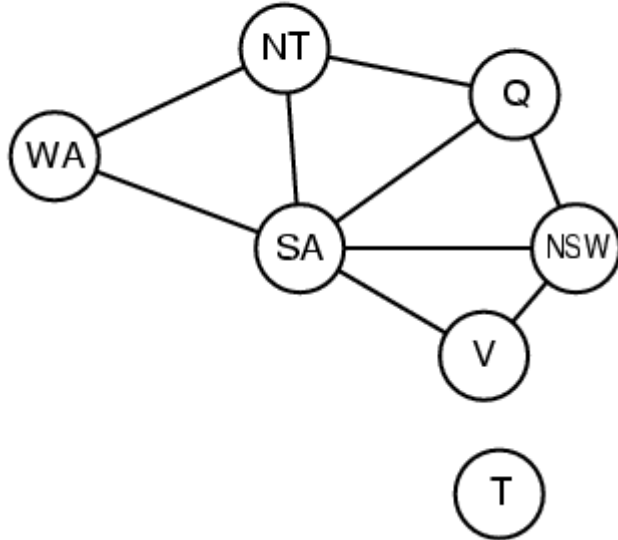
6. Distribute $\vee$ over $\wedge$:

$$[\textit{Animal}(F(x)) \vee \textit{Loves}(G(x),x)] \wedge [\neg \textit{Loves}(x,F(x)) \vee \textit{Loves}(G(x),x)]$$

Resolution proof: definite clauses



Hard matching: The Australian Map Coloring



WA: Western Australia

NT: Northern Territory

SA: South Australia

Q: Queensland

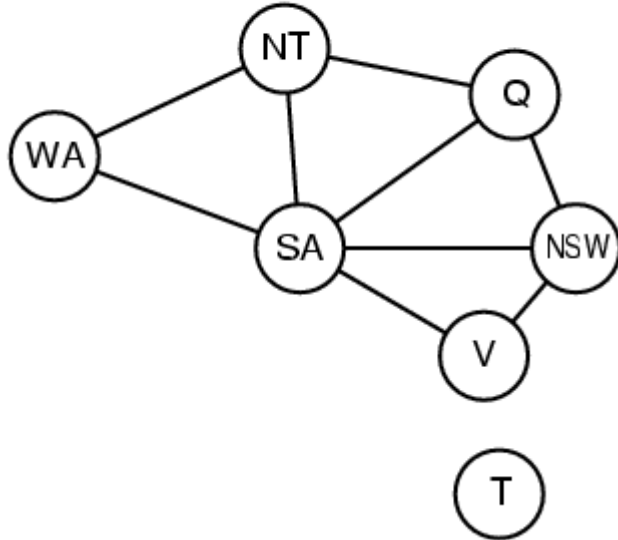
NSW: New South Wales

V: Victoria

T: Tasmania

- **graph representation of the Australian states and territories.**
- **Constraint Satisfaction Problem (CSP)**

Hard matching example



$Diff(wa,nt) \wedge Diff(wa,sa) \wedge Diff(nt,q) \wedge$
 $Diff(nt,sa) \wedge Diff(q,nsw) \wedge Diff(q,sa) \wedge$
 $Diff(nsw,v) \wedge Diff(nsw,sa) \wedge Diff(v,sa) \Rightarrow$
 $Colorable()$

$Diff(Red,Blue) \quad Diff(Red,Green)$
 $Diff(Green,Red) \quad Diff(Green,Blue)$
 $Diff(Blue,Red) \quad Diff(Blue,Green)$

- $Colorable()$ is inferred iff the CSP has a solution
- CSPs include 3SAT as a special case, hence matching is NP-hard

3-Satisfiability:

- Conjunctive Normal Form (CNF)
- Disjunctive Clauses
- Exactly Three Literals